

Code Layout and Reusability

FitBuddy AI Fitness Plan Generator

A map of the supplied application modules and their responsibilities.

Project structure

File or folder	Responsibility
app/main.py	FastAPI setup, static file mounting, route registration and table creation.
app/routes.py	Browser form handling, plan/feedback actions, coach pages and JSON API routes.
app/config.py	Environment-backed application and AI settings.
app/database.py and app/models.py	SQLite engine/session setup and user/plan record definitions.
app/schemas.py	Validation rules for incoming profile and feedback values.
app/services/ai.py	Gemini client, demo response path, prompts and plan revision.
app/templates/	Jinja page templates for profile, results and coach views.
app/static/styles.css	Responsive presentation layer.
README.md and requirements.txt	Setup instructions and dependency constraints.

Reusability principles

- Keep route handlers focused on request/response coordination.
- Centralize configuration so model IDs and database location can change without editing feature code.
- Keep validation rules close to the request schema and persistence definitions in the database layer.
- Keep AI provider behavior separate from page rendering so demo and live modes share the same user flow.

Sharing boundary

The source .gitignore excludes .env and *.db. Preserve those exclusions when packaging the project; a database file is runtime data, not a source-code example.